

Command Line Crash Course

October 26, 2025

In this assignment you will learn about important bash commands and essential command line utilities. Section 1 is an introduction, section 2 explains the assignment, and the subsequent sections give you important information.

1 Bash vs Command Line Utilities

You know you use the command line on Linux to do stuff. It usually doesn't matter what is a Bash command, and what is a command line utility, but it's important that you're aware of the difference.

1.1 Bash

Bash is a programming language. It can do many things you are familiar with, such as for loops and if statements.

```
root@createTree:~# echo "this is bash!"  
this is bash!
```

Figure 1: Bash!

But Bash is a special programming language. Bash was designed to allow you to directly control Unix-like operating systems, such as Linux. Typically when you think about using Linux or other Unix-like operating systems, you imagine typing commands into black window. This black window with a blinking cursor - that's a Bash shell. The shell is the interactive prompt that you type commands into.

Note, you can also write Bash commands into a file much like you would a Python or JavaScript program. These files are referred to as scripts and are given the `.sh` extension.

There are many other languages like Bash, such as **zsh** **ksh**, and they are all primarily used as shells for controlling your *nix computer.

Open a Bash shell and type *help* to see a full list of what Bash can do. See figure 2

1.2 Common Utilities

Bash cannot do everything. After looking through the output of the **help** command you will see some important things are missing, such as **mkdir** and **mv**. These other essential programs are referred to as "system utilities", and they are installed somewhere on your computer. You can use the **which** program to locate them. In figure 3 you see that **mkdir** can be found in the `/usr/bin` directory.

```

root@createTree:~# help
GNU bash, version 5.2.37(1)-release (x86_64-pc-linux-gnu)
These shell commands are defined internally.  Type 'help' to see this
list.
Type 'help name' to find out more about the function 'name'.
Use 'info bash' to find out more about the shell in general.
Use 'man -k' or 'info' to find out more about commands not in this li
t.

A star (*) next to a name means that the command is disabled.

job_spec [&]                history [-c] [-d offset] [n] or>
(( expression ))            if COMMANDS; then COMMANDS; [ e>
. filename [arguments]      jobs [-lnprs] [jobspec ...] or >
:                             kill [-s sigspec | -n signum | >
_

```

Figure 2: Output of help

```

root@createTree:~# which mkdir
/usr/bin/mkdir

```

Figure 3: Where is mkdir?

2 Your Assignment

Your assignment is to create the directory structure shown in figure 5. Make sure that the md5sums of the files match the md5sums shown in 4. If you can do this, you understand the basics of using the command line.

```

root@createTree:~/class# find . -type f | xargs -I {} -n1 md5sum {} ;
2ae4342873f6dc12a664579e2ec5fd0d ./first/dir1/HelloWorld.java
552a6cdcadea5f1ee8d7d6a69620e7f4 ./third/text/poem.txt
1381dc7af94d3fa1ff405976f71f86fa ./second/b/hello.py
54e5252969844975ef38bb95d3e997c0 ./second/a/hello.sh

```

Figure 4: md5sum for each of the files.

```

root@createTree:~/class# tree /root
/root
├── class
│   ├── first
│   │   └── dir1
│   │       └── HelloWorld.java
│   ├── second
│   │   ├── a
│   │   │   └── hello.sh
│   │   ├── b
│   │   │   └── hello.py
│   └── third
│       └── text
│           └── poem.txt

```

Figure 5: tree command showing the directory structure

3 Commands

Play around with the following commands in your terminal until you are comfortable with them.

3.1 pwd

The **pwd** command tells you your present working directory. It's the folder you're currently in on your computer.

```

1 root@machine:~$ pwd
2 /root

```

3.2 cd

The **cd** command is used for changing your directory.

```

1 root@machine$ cd /
2 root@machine$ pwd
3 /
4 root@machine$ cd $HOME
5 root@machine$ pwd
6 /root
7 root@machine$ cd
8 # cd with no arguments is the same as cd $HOME
9 root@machine$ pwd

```

```
10 /root
11 root@machine$ cd /usr/local/bin
12 root@machine$ pwd
13 /usr/local/bin
14 #
```

3.3 mkdir

```
1 root@machine$ cd $HOME
2 root@machine$ mkdir class
3 root@machine$ cd class
4 root@machine$ pwd
5 /root/class
6 root@machine$ mkdir first
7 root@machine$ cd first
8 root@machine$ pwd
9 /root/class/first
10 root@machine$ mkdir dir1
11 root@machine$ cd /root/class
12 root@machine$ mkdir -p second/{a,b}
13 root@machine$ mkdir -p third/text
14 root@machine$ cd $HOME
```

3.4 ls

The **ls** command is for listing the files in your current directory. You can give different flags to the command to change the way it outputs information.

```
1 root@machine$ cd
2 root@machine$ ls
3 class
4 root@machine$ ls class
5 first second third
6 root@machine$ ls -R
7 # alot of information about all of the directories you've created.
8 root@machine$ ls /
9 bin dev home ... and many more directories are shown
```

3.5 cp

The **cp** command is used for copying files.

```
1 root@machine$ cd
2 root@machine$ pwd
3 /root
4 root@machine$ mkdir junk
5 root@machine$ ls
6 class junk
7 root@machine$ cd junk
```

```
8 root@machine$ echo "hi" > hi.txt
9 root@machine$ ls
10 hi.txt
11 root@machine$ cp hi.txt hiagain.txt
12 root@machine$ ls
13 hi.txt hiagain.txt
14 root@machine$ pwd
15 /root/junk
```

Note that you copied hi.txt to a file called hiagain.txt. **cp** can also be used for copying directories.

```
1 root@machine$ cd
2 root@machine$ pwd
3 /root
4 root@machine$ cp -r junk junk_copy
5 root@machine$ ls
6 class junk junk_copy
```

Note that you now have a copy of the junk directory.

3.6 cat

cat is used for “concatenating” files. In other words, cat is used for combining files. However, it is also commonly used for quickly viewing the contents of a file.

```
1 root@machine$ cd /root/junk
2 root@machine$ cat hi.txt
3 hi
4 root@machine$ cat hi.txt hiagain.txt
5 hi
6 hi
7 root@machine$ ls
8 hi.txt hiagain.txt
9 root@machine$ cat hi.txt hiagain.txt > combined.txt
10 root@machine$ ls
11 combined.txt hi.txt hiagain.txt
12 root@machine$ cat combined.txt
13 hi
14 hi
```

3.7 rm

The **rm** command is for deleting files and directories.

```
1 root@machine$ cd /root/junk
2 root@machine$ ls
3 hi.txt hiagain.txt
4 root@machine$ rm hi.txt
5 root@machine$ ls
6 hiagain.txt
7 root@machine$ rm hiagain.txt
8 root@machine$ ls
```

```
# nothing
```

To delete directories, you use the `r` flag.

```
root@machine$ cd
root@machine$ ls
class junk junk_copy
root@machine$ rm junk
# error message
root@machine$ rm -r junk
root@machine$ ls
class junk_copy
root@machine$ rm -r junk_copy
root@machine$ ls
class
```

3.8 apt install

In this class we are focusing on Debian Linux. To install packages on Debian Linux, you use the **apt** package manager. Let's install the **tree** program.

```
root@machine$ tree /root
# error message. tree is not installed
root@machine$ apt install tree
# tree will install
```

3.9 tree

The **tree** command gives you output similar to **ls -R**, but in a nicer format.

```
root@machine$ tree /root
/root
|_ class
|   |_ first
|       |_ dir1
|   |_ second
|       |_ a
|       |_ b
|   |_ third
|       |_ text
```

3.10 md5sum

The **md5sum** command is an important command that gives you a hash of a file. It is used to quickly determine if 2 files are the same. Two files might *appear* to be the same, but one could be a virus and the other one not. To check, you will compare the md5sum of a good file to a suspicious one. If they match, you know the file is good. If not, you can guess the suspicious one is broken or a virus.

```
root@machine$ md5sum /usr/bin/python3
643699f374b7ef7f1bbf2ef3d1867d99 /usr/bin/python3
```

In this assignment, I'm asking you to create some files and verify that they match the files I asked you to create. A quick way to make sure that your file is the same as the one I want you to make, is to use the `md5sum` command.

4 Files To Create

As part of the assignment I've asked you to create some files. Below are screenshots of what the files should look like. Use `vim` to create the files and verify the `md5sum` using the `md5sum` command. Compare your sum to the sum I showed in the beginning of this document.

In the pictures I use **cat** to print out the file contents so you can see what you need to type.

If your `md5sums` don't match mine, you probably have a space wrong or other minor error.

```
root@createTree:~/class/first/dir1# cat HelloWorld.java
public class HelloWorld {
    public static void main(String[] args){
        System.out.println("Hi");
    }
}
```

Figure 6: HelloWorld.java

```
root@createTree:~# cat class/second/a/hello.sh
#!/usr/bin/bash
echo "hi"
```

Figure 7: hello.sh

```
root@createTree:~# cat class/second/b/hello.py
#!/usr/bin/python3
print("hi")
```

Figure 8: hello.py